

Reproducing “The Case for Learned Index Structures”

A Python study of B-Tree vs. Recursive Model Index (RMI) on three key distributions

Vikas · B.Tech Computer Science (3rd year), SRM Institute of Science and Technology, Ramapuram

Code and data: github.com/Vikas-07-cr/learned-index-repro

Abstract

Kraska et al. (SIGMOD 2018) proposed replacing the B-Tree, the classic database index, with machine-learning models that predict where a key sits in a sorted array. I reproduced the core idea in Python: a B-Tree and binary-search baseline, a single linear model, and a two-stage Recursive Model Index (RMI) with per-model error bounds. All indexes were tested for correctness and benchmarked on uniform, lognormal and clustered keys with up to 1,000,000 entries. The main size claim reproduces strongly: an RMI with 1,000 models occupied about 23 KB, roughly 1,000 times less than an estimated 23.9 MB B-Tree. The speed claim reproduces only partly: learned indexes beat the B-Tree on uniform keys, but not on lognormal or clustered keys, and plain binary search was fastest in this Python implementation. On clustered keys, adding more models did not reduce the search window, which points to the first-stage router as the bottleneck.

1. Introduction

A database index answers one question: where in a sorted list is this key? A B-Tree answers it by walking down a tree of pages. The key observation of the paper is that a sorted list of keys is a cumulative distribution function (CDF): the position of a key is approximately $F(\text{key})$ times N . A model that learns F can predict the position directly, and a short binary search around the prediction removes the remaining error.

This project asks three questions. (1) Can a small model replace a B-Tree and still return correct answers? (2) How much memory does it save? (3) Is it faster, and on which key distributions does it fail?

2. Method

Baselines. A B-Tree (the Python BTrees package, OOBTree) and plain binary search over the sorted list. B-Tree size is *estimated* with a textbook model (16 bytes per entry, 69% page fill, fan-out 32), because pickling the full tree exceeds Python's recursion limit.

Single linear model. A least-squares line maps key to position. The minimum and maximum prediction errors over all stored keys define a search window, so every stored key is found by construction. Lookups predict, clamp the window to the array, and binary-search inside it.

Two-stage RMI. Stage 1 is one linear model that routes a key to one of M stage-2 models ($M = 10, 100, 1,000, 10,000$). Each stage-2 model is trained on the keys routed to it, using closed-form least squares computed for all models at once, and stores its own error bounds. Routing uses identical arithmetic at training and lookup time, which keeps the guarantee exact.

Data and measurements. Three synthetic key sets of 100,000 and 1,000,000 keys: *uniform* (evenly spread), *lognormal* (skewed with a long tail) and *clustered* (20 tight groups with large gaps). For each index I recorded lookup time per query (Python loop, mean of 3 repeats over 30,000 random stored keys), vectorised batch lookup time, index size, build time, prediction error and average search window. An 11-test suite checks that every stored key is found at its true position and that missing keys return -1.

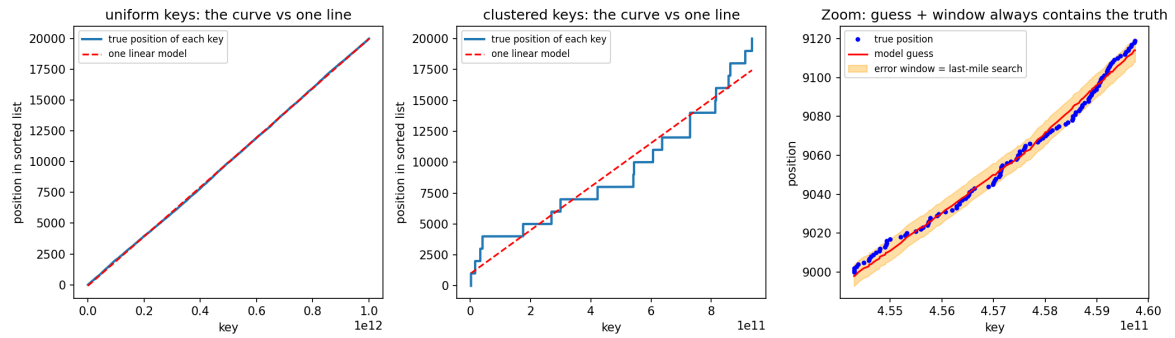


Figure 1. Why it works. A sorted key list is a curve; a line fits the uniform curve almost exactly but the clustered curve poorly. Right: the model's guess plus its error window always contains the true position.

3. Results

3.1 Size. Model parameters are tiny compared with a B-Tree (Table 1, Figure 2). With 1,000 models the RMI needs 23.5 KB against an estimated 23.9 MB for the B-Tree, a factor of about 1,000. Even with 10,000 models (234 KB) it is about 100 times smaller.

Index (1M keys)	Size
B-Tree (estimated)	23.9 MB
Binary search (no extra structure)	0
Single linear model	40 bytes
RMI, 10 models	0.27 KB
RMI, 100 models	2.4 KB
RMI, 1,000 models	23.5 KB
RMI, 10,000 models	234 KB

Table 1. Index size at 1,000,000 keys.

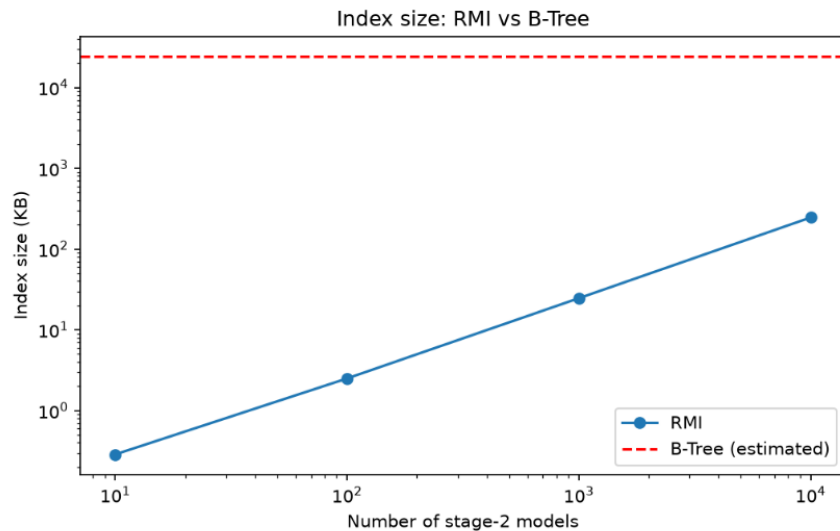


Figure 2. Index size grows linearly with the number of models, but stays far below the B-Tree (dashed line).

3.2 Search window and prediction quality. The width of the last-mile search window depends heavily on the key distribution (Table 2, Figure 3). On uniform keys it shrinks steadily from about 375 positions to about 12. On lognormal keys it stays near 400,000 until 1,000 models are used, then falls to about 1,000 at 10,000 models. On clustered keys it stays between about 62,000 and 79,000 positions at every model count.

Avg. window (positions)	10 models	100	1,000	10,000
uniform	375	119	37	12
lognormal	406,248	393,671	19,093	1,030
clustered	79,007	62,374	68,189	67,088

Table 2. Average search window of the RMI at 1,000,000 keys (computed exactly; data are seeded).

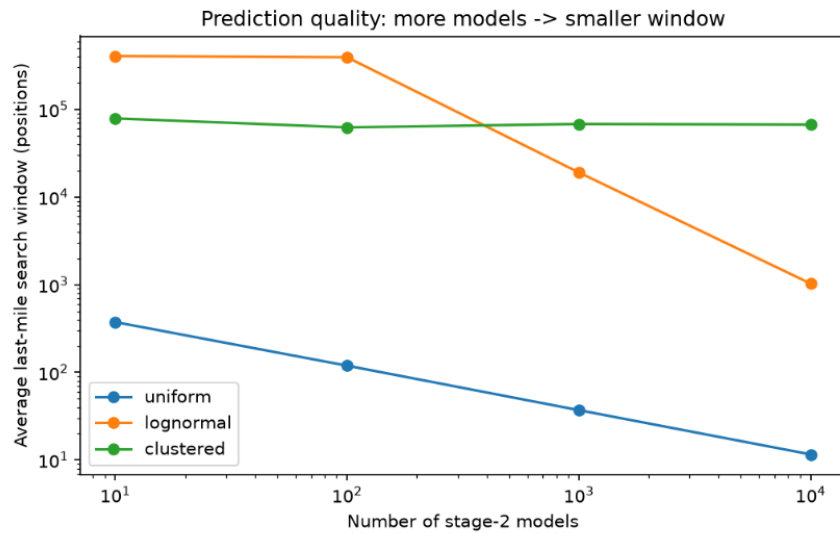


Figure 3. More models shrink the window on uniform and lognormal keys, but not on clustered keys.

3.3 Lookup speed. Table 3 and Figure 4 show mean time per lookup in a Python loop at 1,000,000 keys. The learned indexes beat the B-Tree on uniform keys (single model 0.91 and RMI 1.15 against 1.75 microseconds) and the single model also beat it on clustered keys. The RMI was slower than the B-Tree on lognormal and clustered keys. Plain binary search was the fastest in all three cases.

Mean lookup (us)	B-Tree	Binary search	1 linear model	RMI (1,000)
uniform	1.75	1.02	0.91	1.15
lognormal	1.24	0.90	1.41	1.67
clustered	1.21	0.85	1.04	1.88

Table 3. Mean lookup time at 1,000,000 keys, rounded values read from the benchmark output (results/results.csv holds the exact numbers).

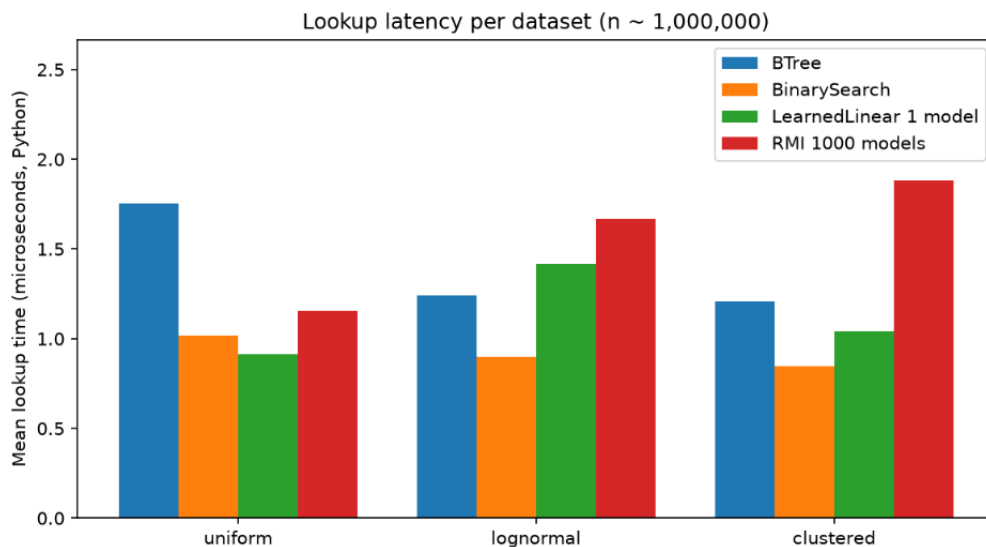


Figure 4. Lookup latency per dataset (Python loop).

3.4 Batch lookups. To remove the Python-loop overhead, I compared a vectorised RMI lookup with NumPy's compiled binary search (Figure 5). The RMI needed about 0.30 to 0.37 microseconds per lookup against about 0.13 to 0.14 for NumPy, so compiled binary search was about 2 to 2.8 times faster even in batch mode.

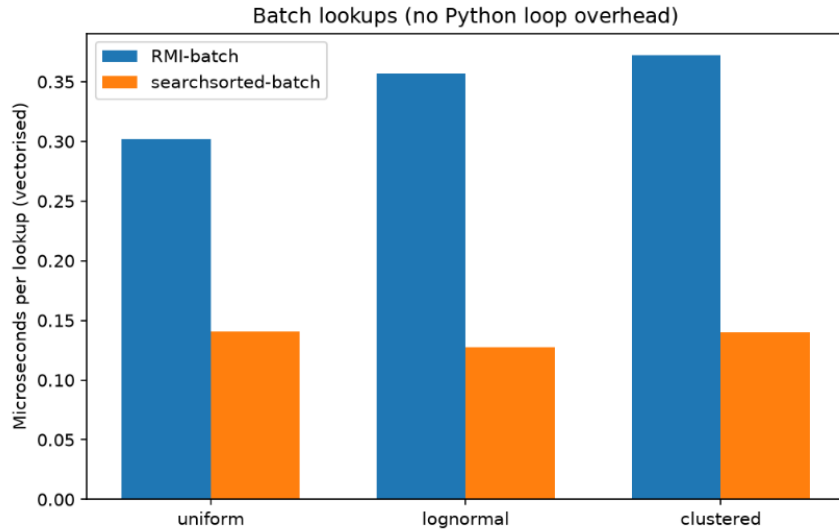


Figure 5. Batch (vectorised) lookups: RMI vs. NumPy binary search.

3.5 Trade-off. Figure 6 plots size against speed. Every learned index sits far to the left of the B-Tree (much smaller) with latency in the same range.

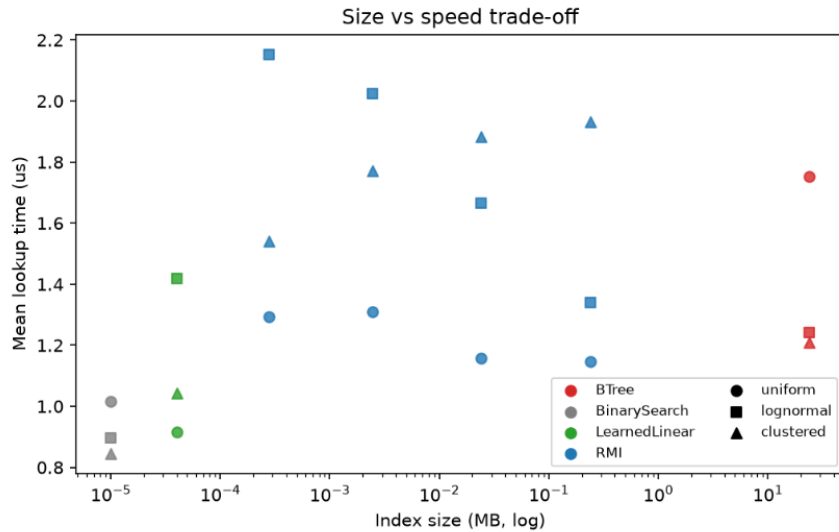


Figure 6. Size vs. speed. Learned indexes (blue, green) are orders of magnitude smaller than the B-Tree (red); binary search (gray) has no extra size.

4. Discussion

What reproduces. The central memory claim holds: a handful of linear models replaced a multi-megabyte tree and every index returned correct positions on every dataset. The speed claim holds only for easy data. This matches the picture in Figure 1: where the key distribution is close to a straight line, a tiny model is accurate; where it is not, the search window grows and the advantage disappears.

The clustered result. Increasing the number of stage-2 models from 10 to 10,000 left the window near 65,000 positions on clustered keys, while it shrank by roughly a factor of 30 on uniform keys and 400 on lognormal keys. A plausible explanation, consistent with the data but not tested here, is that the single linear stage-1 router sends the keys of several clusters (and the large gaps between them) to the same

few stage-2 models, so extra models stay unused. The original paper uses small neural networks at stage 1; testing that is the natural next step.

Why speed favours the baselines here. The B-Tree and the bisect search run as compiled C, while the model arithmetic runs in Python. Even in batch mode, compiled binary search over 1M in-memory keys is hard to beat. The learned index is most attractive when memory or cache footprint is the limiting factor, which is where it was clearly best.

5. Limitations and future work

- Python overhead; a C/C++ or Numba implementation is needed to test the speed claim fairly.
- B-Tree size is an estimate from a standard model, not a measured value.
- Static data only: no inserts or updates (as in the paper's basic design).
- Synthetic data only; real datasets such as the SOSD benchmark would strengthen the study.
- Linear models only; a neural-network stage 1, a delta buffer for inserts, and the tuning of model count per dataset are the planned extensions.

References

- [1] T. Kraska, A. Beutel, E. H. Chi, J. Dean, N. Polyzotis. *The Case for Learned Index Structures*. SIGMOD 2018. arXiv:1712.01208.
- [2] R. Marcus et al. *Benchmarking Learned Indexes (SOSD)*. PVLDB 2020.